Digitale Codierungen | DigCod

Zusammenfassung

INHALTSVERZEICHNIS

1. Stellenwertsystem ... 2
1.1. Konversion ... 2
1.2. Nachkommastellen ... 2
1.3. Subtraktion durch Addition ... 2

2. Dualsystem ... 3
2.1. Arithmetik ... 3
2.1.1. Komplementbildung (Zweierkomplement) ... 3
2.1.2. Subtraktion ... 3
2.1.3. Addition ... 3
2.1.4. Multiplikation ... 3
2.1.5. Division ... 3
2.2. Wertebereich ... 3
2.3. Bit ... 4
2.4. Präfixe ... 4
2.4.1. Multiplikation/Division ... 4

3. Codierungen ... 4
3.1. Warum reicht eine einzige Zahlendarstellung nicht aus? ... 4
3.2. Vorzeichen ... 5
3.2.1. Betrag mit Vorzeichen ... 5
3.2.2. (b-1) Komplement / Einerkomplement ... 6
3.2.3. (b) Komplement / Zweierkomplement ... 6
3.2.4. Exzess-Codierung (Bias-Code) ... 6
3.3. Gleitkommazahlen ... 7
3.3.1. Fixkommazahl ... 7
3.3.2. Allgemeiner Wertebereich ... 8
3.3.3. Auflösung ... 9
3.3.4. Arithmetik ... 9
3.4. Gleitkomma ... 9
3.4.1. Sonderfälle ... 10
3.4.2. Präzision ... 10
3.4.3. Addition ... 10
3.4.4. IEEE 754 - Single vs. Double ... 11
3.5. Text ... 11
3.5.1. ASCII ... 11
3.5.2. Unicode ... 12

4. Qubit ... 12

5. CPU ... 13

1. STELLENWERTSYSTEM

Begriff	*Bedeutung*
Basis (Radix)	$E \in \mathbb{N}, R \geq 2$
Ziffernmenge	$T_R = \{0, 1, \ldots, R-1\}$
Darstellung einer Zahl (Polynom)	$N_R = \sum_{i=0}^{n} d_i R^i$
Stellenwertigkeit an der Position i	$d_i \in Z_R, R^i$

Beispiele

Dezimalsystem – $R = 10$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ – $(110)_{10} = (110)_{10}$	Oktalsystem – $R = 8$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7\}$ – $(110)_8 = (72)_{10}$
Dualsystem – $R = 2$ – $Z_{10} = \{0, 1\}$ – $(110)_2 = (6)_{10}$	Hexadezimalsystem – $R = 16$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ – $(110)_8 = (48)_{10}$

1.1. KONVERSION

Dezimal- zu Dualsystem:
- Rechts shift ist eine Division durch 2
- Es entsteht nur dann ein Rest, wenn die Bitstelle $2^0 = 1$ ist.

Beispiel: **25**
Resultat: **11001**

Zähler	*Nenner*	*Resultat*	*Rest*
25	2	12	**1**
12	2	6	**0**
6	2	3	**0**
3	2	1	**1**
1	2	0	**1**

↑

Dual- zu Dezimalsystem bei Nachkommastellen:
- Links shift ist eine Multiplikation mit 2

Beispiel: **0.1875**
Resultat: **0.0011**

a	*b*	*Resultat*	*Rest*
0.1875	2	0.375	**0**
0.375	2	0.75	**0**
0.75	2	1.5	**1**
0.5	2	1	**1**

↓

1.2. NACHKOMMASTELLEN

Erweiterung der Darstellung einer Zahl: $N_R = d_n R^n + \ldots + d_{10} R^0 + d_{-1} R^{-1} + \ldots + d_{-m} R^{-m}$

Beispiel: $(101.01)_2 = 1 * 2^2 + 0 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2} = 4 + 1 + \frac{1}{4} = 5.25$

1.3. SUBTRAKTION DURCH ADDITION

Beispiel: $753 + 247 = 1000$

Bei **1000** einen Überlauf (**mod 1000**): $753 + 247 \equiv 0 \Leftrightarrow 753 \equiv -247 \bmod 1000$

Dann wäre $620 - 247 \equiv 620 + 753 = 1373 \equiv 373 \bmod 1000$

2. DUALSYSTEM

2.1. ARITHMETIK

2.1.1. Komplementbildung (Zweierkomplement)

1) Rechnen in n Bit $\Rightarrow \bmod 2^n$
   - Übertrag aus dem MSB wird verworfen (gerechnet wird in $\mathbb{Z}_{2^n}$)
2) $-b$ als Zweierkomplement
   - Additives Inverses von $b \bmod 2^n$ ist: $-b \equiv 2^n - b \bmod 2^n$
   - Praktische Berechnung:
     - Bits invertieren: $\sim b$
     - $+1$ addieren
     - $2K(b) = \sim b + 1$

2.1.2. Subtraktion

Subtraktion wird durch Addition des Komplements ersetzt

$$a - b \equiv a + 2K(b) \bmod 2^n$$

Beispiel 1 : $13 - 5$

8 Bit, wobei $13 = 0000\ 1101, 5 = 0000\ 0101$
Zweierkomplement von 5:
- invertieren: $1111\ 1010$
- $+1 : 1111\ 1011$

Addieren: $0000\ 1101 + 1111\ 1011 = 1\ 0000\ 1000$

MSB-Übertrag weg: $0000\ 1000 = 8$

Also: $13 - 5 = 8$

2.1.3. Addition

TODO:
signed, unsigned

2.1.4. Multiplikation

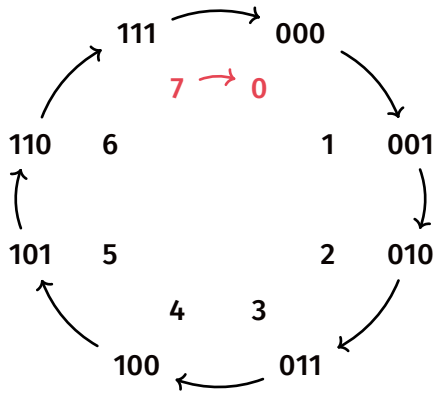
TODO:
signed, unsigned

2.1.5. Division

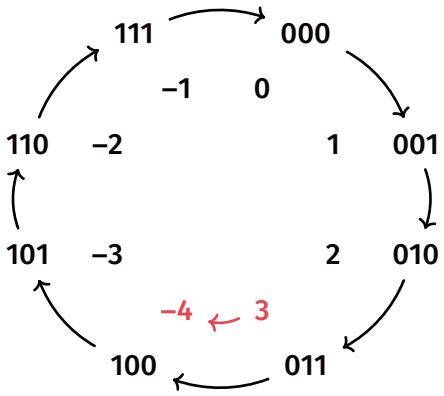
TODO:
signed, unsigned

2.2. WERTEBEREICH

Graphische Veranschaulichung für Wortbreite von 3 Bit



unsigned: Überlauf von **7** auf **0**



signed: Überlauf von **3** auf **−4**

2.3. BIT

Begriff	*Bedeutung*
set bit (gesetztes Bit)	**1**
cleared bit (gelöschtes Bit)	**0**
LSB	Least Significant Bit (Bit **0**)
MSB	Most Significant Bit (Bit $n-1$) in einer n-stelligen Binärzahl
Nibble	Binärzahl mit 4 Bit
Oktett	Binärzahl mit 8 Bit
Byte	Oktett

2.4. PRÄFIXE

Binär	*Näherung*	*Binärpräfix*	*Dezimal*	*Dezimalpräfix*
2^{10}	$1.024 \cdot 10^3$	Ki - Kibi	10^3	K - Kilo
2^{20}	$1.049 \cdot 10^6$	Mi - Mebi	10^6	M - Mega
2^{30}	$1.074 \cdot 10^9$	Gi - Gibi	10^9	G - Giga
2^{40}	$1.100 \cdot 10^{12}$	Ti - Tebi	10^{12}	T - Tera
2^{50}	$1.126 \cdot 10^{15}$	Pi - Pebi	10^{15}	P - Peta
2^{60}	$1.153 \cdot 10^{18}$	Ei - Exbi	10^{18}	E - Exa

2.4.1. Multiplikation/Division

1) Umformen in Potenzschreibweise
2) Addition der Exponenten
3) Umformen in Präfixschreibweise

Beispiel: $128K \cdot 64M = 2^7 \cdot 2^{10} \cdot 2^6 \cdot 2^{20} = 2^{17+26} = 2^{43} = 2^3 \cdot 2^{40} = 8T$

3. CODIERUNGEN

Erst wenn man die Codierung kennt, kann man daten richtig interpretieren.

3.1. WARUM REICHT EINE EINZIGE ZAHLENDARSTELLUNG NICHT AUS?

- Negative Zahlen
  - Vorzeichen-Bit
  - Einerkomplement
  - Exzess-Code

- Sehr grosse und kleine Zahlen
  - Fixkomma ist unflexibel
  - 334 Milliarden + 0.00001
- Genauigkeit
  - 0.1 ist im Dualsystem nicht exakt darstellbar
  - Rundungsfehler unvermeidbar
- Nicht nur Zahlen
  - Textdarstellung
  - Zeichenkodierungen

3.2. VORZEICHEN

Jede Codierung optimiert eine andere Eigenschaft

- Betrag mit Vorzeichen: Intuition
- Einerkomplement: Einfache Negation
- Zweierkomplement: Arithmetische Effizienz
- Exzess: Vergleichbarkeit/Sortierung

Binär	*Betrag & V.*	*EinerK.*	*ZweierK.*	$C_{ex,-8,4}$
0000	0	0	0	-8
0001	1	1	1	-7
0010	2	2	2	-6
0011	3	3	3	-5
0100	4	4	4	-4
0101	5	5	5	-3
0110	6	6	6	-2
0111	7	7	7	-1
1000	0	-7	-8	0
1001	-1	-6	-7	1
1010	-2	-5	-6	2
1011	-3	-4	-5	3
1100	-4	-3	-4	4
1101	-5	-2	-3	5
1110	-6	-1	-2	6
1111	-7	0	-1	7

3.2.1. Betrag mit Vorzeichen

Erstes Bit signalisiert, ob Zahl positiv (**0**) oder Negativ (**1**) ist.

Problem: Bekannte Rechenregeln funktionieren nicht.

$$-19_{10} \quad + 1_{10} \quad = -18_{10}$$
$$1001\ 0011_2 + 0000\ 0001_2 = 1001\ 0100_2 = -20_{10}$$

Beispiel 2 : **Codierung**

Gegeben	*Dezimal → Binär*	*Ergebnis*
$x = -3_{10}$	$3_{10} = x011_2$	$-3_{10} = 1011_2$

Beispiel 3 : **Decodierung**

Gegeben	*Binär → Dezimal*	*Ergebnis*
$x = 1001_2$	$x001_{10} = 1_2$	$1001_2 = -1_{10}$

3.2.2. (b-1) Komplement / Einerkomplement

Von allen bits wird das Komplement gebildet.

Problem: $\mathbf{0000\ 00000_2 = 1111\ 1111_2 = 0_{10}}$

Beispiel 4 : **Codierung**

Gegeben	*Dezimal → Binär*	*Komplement*	*Ergebnis*
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$-5_{10} = 1010_2$

Beispiel 5 : **Decodierung**

Gegeben	*Komplement*	*Binär → Dezimal*	*Ergebnis*
$x = 1011_2$	$1011_2 \Rightarrow 0100_2$	$0100_2 = 4_{10}$	$1011_2 = -4_{10}$

3.2.3. (b) Komplement / Zweierkomplement

Nach der Komplementbildung wird **1** addiert.

Beispiel 6 : **Codierung**

Gegeben	*Dezimal → Binär*	*Komplement*	*+1*	*Ergebnis*
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$0101_2 \Rightarrow 1011_2$	$-5_{10} = 1011_2$

Beispiel 7 : **Decodierung**

Gegeben	*−1*	*Komplement*	*Binär → Dezimal*	*Ergebnis*
$x = 1111_2$	$1111_2 \Rightarrow 1110_2$	$1110_2 \Rightarrow 0001_2$	$0001_2 = 1_{10}$	$1111_2 = -1_{10}$

3.2.4. Exzess-Codierung (Bias-Code)

Darstellung vorzeichenbehafteter Zahlen durch **Verschiebung des Wertebereichs**.

Statt negativer Zahlen wird ein **Offset (Bias)** addiert

Begriff 1 : $C_{ex,-B,n}(x)$

C_{ex} = Exzess-Codierung
$-B$ = Bias (negativer Überhang zur 0)
n = Länge der binären Schreibweise
x = zu codierende Zahl

Gespeichert wird: $c = x + B$

Decodierung: $x = c - B$

Gegeben	Codierung	Decodierung
Basis $b = 2$ Wortbreite n Bias B typischerweise: $B = 2^{n-1} - 1$	$C_{ex,-B,n}(x) = x + B$ mit $x \in [-B, 2^n - 1 - B]$	$x = c - 127$

Beispiel 8 : **Codierung**

Gegeben

Wortbreite: $n = 8$ Bit
Bias: $B = 2^7 - 1 = 127$
Zu codierende Zahl: $x = -5$

Bias addieren

$c = x + B = -5 + 127 = 122$

Dezimal → Binär

$122_{10} = 01111010_2$

Ergebnis

$C_{ex,-127,8}(-5) = 01111010_2$

Beispiel 9 : **Decodierung**

Gegeben

Wortbreite: $n = 8$ Bit
Bias: $B = 2^7 - 1 = 127$
Bitmuster: $C_{ex,-127,8} = 01111010_2$

Binär → Dezimal

$01111010_2 = 122_{10}$

Bias subtrahieren

$x = c - B = 122 - 127 = -5$

Ergebnis

$01111010_2 \Rightarrow -5$

3.3. GLEITKOMMAZAHLEN

3.3.1. Fixkommazahl

Skalierte Ganzzahl

Pro	Contra
Einfache Implementierung	Fester Wertebereich

Deterministische Genauigkeit	Feste Auflösung
Keine Exponentendarstellung	Überlauf bei grossen Zahlen
Schnelle Berechnung	Ungeeignet für stark unterschiedliche Grössenordnungen

Begriff 2 : $C_{FK,k,n}(x) = x \cdot 2^k$

C_{FK} = Fixkomma-Codierung
k = Anzahl Nachkommabits
n = Länge der binären Schreibweise - daraus ergibt sich die Anzahl der Vor-Kommastellen: $n - k$
x = zu codierende Zahl
I = Ganzzahl

Beispiel 10 : **Codierung**

Gegeben

Wortbreite: $n = 8$ Bit
Nachkommabits: $k = 4$
Zu codierende Zahl: $x = 3.25$

Skalieren

$I = x \cdot 2^k = 3.25 \cdot 16 = 52$

Dezimal → Binär

$52_{10} = 00110100_2$

Ergebnis

$C_{FK,4,8}(3.25) = 00110100_2$

Beispiel 11 : **Decodierung**

Gegeben

Wortbreite: $n = 8$ Bit
Nachkommabits: $k = 4$
Bitmuster: $C_{FK,4,8}(3.25)$

Binär → Dezimal

$00110100_2 = 522_{10} = I$

Skalieren

$x = \frac{I}{2^k} = \frac{52}{16} = 3.25$

Ergebnis

$00110100_2 \Rightarrow 3.25$

3.3.2. Allgemeiner Wertebereich

Wertebereich bei n Bit und k Nachkommabits

	Ganzzahlbereich	Reeller Bereich
unsigned	$I \in [0, 2^n - 1]$	$x \in \left[0, \frac{2^n - 1}{2^k}\right]$
signed	$I \in \left[-2^{n-1}, 2^{n-1} - 1\right]$	$x \in \left[-\frac{2^{n-1}}{2^k}, \frac{2^{n-1} - 1}{2^k}\right]$

Beispiel 12

Gegeben

Wortbreite: $n = 8$ Bit
Nachkommabits: $k = 3$
Darstellung im Zweierkomplement (= inkl. Vorzeichen)

Ganzzahlbereich

$I \in \left[-2^{8-1}, 2^{8-1} - 1\right] = [-128, 127]$

Reeller Bereich

$$x = \frac{I}{2^k} = \frac{I}{8}$$

$$x_{\text{min}} = \frac{-128}{8} = -17$$

$$x_{\text{max}} = \frac{127}{8} = 15.875$$

Ergebnis

$x \in [-16, 15.875]$

3.3.3. Auflösung

Kleinster darstellbarer Schritt: $\Delta x = 2^{-k}$

Absoluter Fehler: $E_{\text{abs}} = \left|x_{\text{korrekt}} - x_{\text{gerundet}}\right|$

Relativer Fehler: $E_{\text{rel}} = \frac{\left|x_{\text{korrekt}} - x_{\text{gerundet}}\right|}{x_{\text{korrekt}}}$

3.3.4. Arithmetik

Addition, Subtraktion und Zweierkomplement sind wie bei Ganzzahlen

- Addiert man zwei Fixkommazahlen $z_0 + z_1$ mit $k_0 > k_1$, so muss z_1 um $k_0 - k_1$ Bits nach rechts geschoben werden.

Multiplikation und Division verschieben das Komma

- Multipliziert man zwei Fixkommazahlen $z_0 \cdot z_1 = z_2$, dann ist $k_2 = k_0 + k_1$

k muss für jede Zahl berücksichtigt werden

- Wird von den meisten Programmiersprachen kaum unterstützt
- Meist von Hand in Kommentaren
- Limitiert die Zahlen auf Bereiche, die zur Compilezeit bekannt sind
- unflexibel und fehleranfällig, aber performant

3.4. GLEITKOMMA

Begriff 3 : $\pm m \cdot 2^e$

m = Mantisse (Signifikand)
e = Exponent

← 32 Bit →		
±	8 – Bit Exponent	23 – Bit Mantisse

$$\pm(1 + \text{Mantisse}) \cdot 2^{\text{Exponent} - 127}$$

3.4.1. Sonderfälle

Fall	*Vorzeichenbit*	*Exponent*	*Mantisse*	*Beispielwert*
Positive Null	0	00000000	00000000000000000000000	+0
Negative Null	1	00000000	00000000000000000000000	−0
Positive Unendlichkeit	0	11111111	00000000000000000000000	$+\infty$
Negative Unendlichkeit	1	11111111	00000000000000000000000	$-\infty$
NaN	0 oder 1	11111111	mindestens ein Bit 1	Nicht darstellbare Werte
Subnormale Zahlen	0 oder 1	00000000	mindestens ein Bit 1	$\approx 1.4 \cdot 10^{-45}$ (positiv)
Normalisierte Zahlen	0 oder 1	alles ausser 00000000 und 11111111	beliebig	Alle regulären Werte

3.4.2. Präzision

Viele Zahlen können nicht präzise dargestellt werden

- $0.1_{10} = 0.0001100110011...$
  - Abbruch nach 23 Bits
  - Beispiel: $0.1 + 0.2 \neq 0.3$
- Präzision für «Single Precision»: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
  - ε ist die kleinste relative Differenz nahe 1

3.4.3. Addition

1) Hidden Bit ergänzen
2) Exponenten vergleichen
   - Wenn unterschiedlich: Bei kleinerer Zahl Mantisse nach rechts schieben
3) Vorzeichen berücksichtigen
   - Gleiche Vorzeichen: Addieren
   - Unterschiedliche Vorzeichen: Subtrahieren
4) Addition/Subtraktion der Signifikanden
5) Falls Carry = 1: Normalisieren
   - Erhöhe Exponent um 1
   - Schiebe Signifikand um 1 nach rechts
6) Hidden Bit entfernen

Beispiel 13 : $x = 1.5, y = 0.75$

1) Hidden Bit ergänzen

$$x' = 0 \mid 0111\ 1111 \mid (1)100\ 0000\ 0000\ 0000\ 0000\ 0000$$
$$y' = 0 \mid 0111\ 1110 \mid (1)100\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Exponent verschieben bei y'

$$x'' = 0 \mid 0111\ 1111 \mid (1)100\ 0000\ 0000\ 0000\ 0000\ 0000$$
$$y'' = 0 \mid 0111\ 1111 \mid (0)110\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Vorzeichen: beide positiv → Addition
2) Addition $z'' = x'' + y''$

$$z'' = 0 \mid 0111\ 1111 \mid (10)010\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Carry = 1: Normalisieren

$$z' = 0 \mid 1000\ 0000 \mid (1)001\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Hidden Bit entfernen

$$z = 0 \mid 1000\ 0000 \mid 001\ 0000\ 0000\ 0000\ 0000\ 0000$$

3.4.4. IEEE 754 - Single vs. Double

Format	*Bits*	*Vorzeichen*	*Exponent*	*Mantisse*	*Bias*
Single	32	1	8	23	−127
Double	64	1	11	52	−1023

- Grösserer Exponent → grösserer Wertebereich
- Grössere Mantisse → höhere Präzision
  - Single Precision: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
  - Double Precision: $\varepsilon \approx 2^{-52} \approx 10^{-16}$

• Interpretation
- Single ≈ **7** signifikante Dezimalstellen
- Double ≈ 15^{-16} signifikante Dezimalstellen

3.5. TEXT

- Text besteht aus einer endlichen Folge von Zeichen: Buchstaben, Zahlen, Satzzeichen usw.
- Alle Zeichen stammen aus einer endlichen Menge Z, dem Zeichensatz (character set)
  - Jedem Zeichen kann eindeutig eine natürliche Zahl zugeordnet werden
- Ein Encoding (character encoding, Zeichenkodierung) ist eine bijektive Funktion E, die jedem Zeichen z eine natürliche Zahl zuordnet
  - $E : Z \to \{0, 1, \ldots, |Z| - 1\}$
  - Text mit n Zeichen $z_0, \ldots, z_{n-1}$ kann als endliche Folge von kodierten Zeichen beschrieben werden

3.5.1. ASCII

American Standard Code for Information Interchange

- Grösse des Zeichensatzes: $2^7 = 128 \to$ **7 Bit** (nicht 8 Bit!)
- 8-Bit-ASCII sind Extended ASCII-Varianten und nicht standardisiert, sondern Codepages – bspw.: – ISO-8859-1 (Latin-1)
  - für westeuropäische Sprachen mit zusätzlichen Buchstaben wie ä, ö, ü, é usw.

  – Windows-1252
  - Microsoft-Codepage – weitgehend wie ISO-8859-1 mit typografischen Zeichen wie €, “ usw.

  – Codepage 437
  - IBM-PC-Zeichensatz mit grafischen Symbolen, Linienzeichen und Sonderzeichen

– Enthält druckbare (darstellbare) Zeichen und (nicht darstellbare) Steuerzeichen (0x00=NUL, 0x07=BEL, …)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	**NUL** 0	SOH 1	STX 2	ETX 3	EOT 4	ENQ 5	ACK 6	BEL 7	**BS** 8	**TAB** 9	**LF** A	VT B	FF C	**CR** D	SO E	SI F
1	DLE 10	DC1 11	DC2 12	DC3 13	DC4 14	NAK 15	SYN 16	ETB 17	CAN 18	EM 19	SB 1A	ESC 1B	FS 1C	GS 1D	RS 1E	US 1F
2	**SP** 20	**!** 21	**“** 22	**#** 23	**$** 24	**%** 25	**&** 26	**‘** 27	**(** 28	**)** 29	***** 2A	**+** 2B	**,** 2C	**–** 2D	**.** 2E	**/** 2F
3	**0** 30	**1** 31	**2** 32	**3** 33	**4** 34	**5** 35	**6** 36	**7** 37	**8** 38	**9** 39	**:** 3A	**;** 3B	**<** 3C	**=** 3D	**>** 3E	**?** 3F
4	**@** 40	**A** 41	**B** 42	**C** 43	**D** 44	**E** 45	**F** 46	**G** 47	**H** 48	**I** 49	**J** 4A	**K** 4B	**L** 4C	**M** 4D	**N** 4E	**O** 4F
5	**P** 50	**Q** 51	**R** 52	**S** 53	**T** 54	**U** 55	**V** 56	**W** 57	**X** 58	**Y** 59	**Z** 5A	**[** 5B	**** 5C	**]** 5D	**^** 5E	**_** 5F
6	**`** 60	**a** 61	**b** 62	**c** 63	**d** 64	**e** 65	**f** 66	**g** 67	**h** 68	**i** 69	**j** 6A	**k** 6B	**l** 6C	**m** 6D	**n** 6E	**o** 6F
7	**p** 70	**q** 71	**r** 72	**s** 73	**t** 74	**u** 75	**v** 76	**w** 77	**x** 78	**y** 79	**z** 7A	**{** 7B	**|** 7C	**}** 7D	7E	**DEL** 7F

3.5.2. Unicode

Globale Zeichennummerierung

– ASCII-kompatibel
– 1 bis 4 Byte pro Zeichen
– häufige Zeichen kurz
– seltene Zeichen länger
– das erste Byte verrät, wie viele Bytes folgen
– Unicode = Codepoint
– UTF-8 = Bytecodierung

4. QUBIT

– Superposition
  – Ein Qubit kann gleichzeitig **0** und **1** repräsentieren. Mehrere Qubits können dadurch alle 2^n möglichen Zustände gleichzeitig darstellen.
  – $|\Psi\rangle = \alpha\ |0\rangle + \beta\ |1\rangle$
  – $P(0) = |\alpha|^2, P(1) = |\beta|^2$
– Beispiel
  – $|\Psi\rangle = \sqrt{0.75}\ |0\rangle + \sqrt{0.25}\ |1\rangle$
  – $P(0) = 0.75, P(1) = 0.25$
– Interferenz
  – α, β sind Amplituden. Zwei Qubits können sich gegenseitig beeinflussen, indem die Amplituden mittels Interferenz verstärkt oder ausgelöscht werden
  – Quantenalgorithmen verändern gezielt die Amplituden:
    – richtige Lösungen werden verstärkt

    - falsche Lösungen werden abgeschwächt
- Verschränkung
  - Verschränkung bedeutet, dass zwei Qubits einen gemeinsamen Zustand besitzen, der sich nicht in zwei unabhängige Einzelzustände zerlegen lässt.
  - Misst man eines der Qubits, ist der Zustand des anderen sofort festgelegt

5. CPU

Gesamtübersicht des CPU Zyklus

1) Fetch: Instruktion aus RAM lesen
2) Decode: Opcode interpretieren
3) Operand-Fetch: Speicherwerte laden
4) Execute: ALU rechnet
5) Writeback: Ergebnis ins Register
6) IP erhöhen